

Day 8 — Introduction to AI Agents

From GPT Responses to Goal-Driven Action

Course

GPT + AI Agents + RAG + n8n + API Integration + Business Automation

Lecture Duration

60 Minutes

Level

Beginner → Early Intermediate

Day 8 Main Goal

Week 1 focused primarily on AI systems that receive information and generate an answer.

For example:

```
Customer Message
  ↓
  GPT
  ↓
Classification
```

Starting in Week 2, students make an important transition:

```
User Goal
  ↓
AI Agent
  ↓
Decide What Information Is Needed
  ↓
Choose Tool
  ↓
Use Tool
  ↓
```

```
Observe Result
↓
Decide Next Step
↓
Final Result
```

An AI agent is more than a chatbot response. In modern agent frameworks, an agent is typically an LLM configured with instructions and tools, with an execution loop that can manage multiple turns, tool calls, guardrails, state, and sometimes delegation to other agents. OpenAI's current Agents SDK describes an agent in these terms and can manage the orchestration loop through an `Agent` and `Runner`.

By the end of Day 8, students should understand:

- What an AI Agent is
- Chatbot vs AI Agent
- Workflow vs Agent
- Agent goals
- Agent instructions
- Tools
- Tool calling
- Actions
- Observations
- The agent loop
- State
- Memory
- Context
- Environment
- Planning
- Decision making
- Stop conditions
- Tool permissions
- Guardrails
- Human approval
- Agent autonomy
- Single-agent architecture
- Multi-agent concept
- When agents are useful
- When agents are unnecessary
- Business use cases
- Agent risks
- How agents connect with APIs, RAG, and n8n

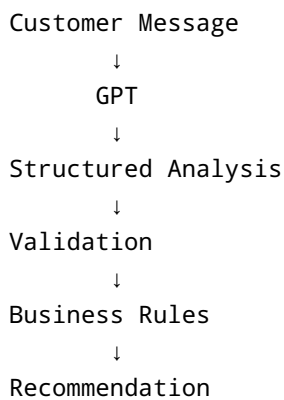
1. Day 8 — 60-Minute Schedule

Time	Topic
0–5 min	Week 1 review
5–12 min	What is an AI Agent?
12–18 min	Chatbot vs workflow vs agent
18–26 min	Core components of an agent
26–34 min	The agent execution loop
34–41 min	Tools and tool calling
41–47 min	State, memory, and context
47–52 min	Guardrails, permissions, and human approval
52–56 min	Business agent examples
56–60 min	Exercise, quiz, recap, homework

2. Opening Review — 0 to 5 Minutes

Start by reminding students what they built in Week 1.

The Day 7 system looked approximately like:



Ask:

Who decides what happens next?

In the Week 1 architecture, most of that decision logic was predetermined by our workflow.

For example:

```
IF category == shipping
→ shipping workflow

IF requires_human_review == true
→ human review
```

Today we introduce systems where the AI can make some of those decisions dynamically.

3. Day 7 vs Day 8

Day 7

We gave GPT a specific task:

Analyze this customer email.

GPT returned:

```
{
  "category": "shipping",
  "priority": "high",
  "requires_external_data": true
}
```

The workflow decided what happened next.

Day 8

Now imagine we tell an AI:

Help this customer resolve their delayed-order problem.

The AI may decide:

```
I need the order number
↓
I already have it
↓
I should check order status
↓
Use Order Tool
↓
Order is shipped
↓
I should check tracking
↓
Use Carrier Tool
↓
Tracking says delayed
↓
I should search shipping policy
↓
Use Knowledge Tool
↓
Generate response
```

This is closer to an **agentic workflow**.

4. What Is an AI Agent? — 5 to 12 Minutes

Use this beginner definition:

An AI Agent is a software system where an AI model can pursue a goal by deciding what steps to take and which available tools to use.

OpenAI's current documentation describes agents as models equipped with instructions and tools; the Agents SDK additionally supports mechanisms such as guardrails, sessions, handoffs, and managed execution.

5. The Simplest Agent Formula

Write this prominently:

```
AI Agent
=
```

```
Model
+
Instructions
+
Goal
+
Tools
+
Context
+
Execution Loop
```

More advanced systems may also include:

```
Memory
Guardrails
Human Approval
State
Handoffs
Tracing
Evaluations
```

6. A Normal LLM Call

Basic LLM:

```
Input
↓
Model
↓
Output
```

Example:

```
User:
"Where is order #123?"

GPT:
"I don't have access to your order system."
```

7. Agent With a Tool

Now:

```
User
↓
Agent
↓
Understand Request
↓
Choose get_order Tool
↓
Order Database
↓
Tool Result
↓
Agent
↓
Customer Answer
```

The crucial difference is:

The model can decide that it needs another capability before answering.

8. A Business Example

User:

What's happening with order #A8293?

Available tool:

```
get_order(order_id)
```

Agent determines:

```
I need live order information.
```

Then requests:

```
{  
  "order_id": "A8293"  
}
```

The application executes the tool.

Tool returns:

```
{  
  "order_id": "A8293",  
  "status": "shipped",  
  "carrier": "Example Carrier"  
}
```

The agent can then generate:

Order A8293 has shipped.

In current OpenAI function-calling workflows, the model requests a function call, the application executes the function, returns the tool output, and the model continues from that result; this loop can continue for multiple tool calls when necessary.

9. Important Point

The model did **not** directly access the order database.

Instead:

```
Model  
↓  
Requests Tool  
↓  
Application Executes Tool  
↓  
Tool Result  
↓  
Model Continues
```

This distinction is extremely important.

10. Model vs Agent

Model

Example:

GPT

Capabilities:

- Understand text
- Generate text
- Analyze
- Classify
- Extract
- Reason over supplied information

Agent

Example:

Customer Support Agent

Might contain:

```
GPT
+
Instructions
+
Order Tool
+
CRM Tool
+
RAG Search
+
Ticket Tool
+
Execution Loop
```

Therefore:

Agent
≠
Model

The model is an important component **inside** the agent.

11. Chatbot vs Agent — 12 to 18 Minutes

This is one of the most important Day 8 distinctions.

Chatbot

Basic architecture:

User
↓
LLM
↓
Answer

Example:

Explain our return policy.

If the policy is already in context, the chatbot can answer.

12. Agent

Agent architecture:

User
↓
Agent
↓
What do I need?
↓
Choose Tool
↓
Get Information

↓
Evaluate
↓
Maybe Use Another Tool
↓
Answer / Action

13. Example Comparison

Customer asks:

Please check where my order is.

Chatbot

Might say:

Please visit your tracking page.

Agent

May perform:

Extract Order ID
↓
Get Order
↓
Get Tracking
↓
Check Shipping Policy
↓
Generate Answer

14. Chatbot Is Not Bad

Important:

Do not tell students:

Agents are always better.

A chatbot is often enough.

If the task is simply:

```
Question
↓
Answer
```

there may be no need for an agent.

15. Workflow vs Agent

This is another critical distinction.

Deterministic Workflow

The developer decides every step.

```
Step 1
Get Email
↓
Step 2
Classify
↓
Step 3
IF billing
↓
Step 4
Billing API
↓
Step 5
Generate Response
```

16. Agent

The model chooses among available actions.

Goal

↓

Agent

- |— Search Policy
- |— Get Order
- |— Get Customer
- |— Create Ticket
- |— Escalate

Depending on the request, it may use different tools in a different order.

OpenAI's orchestration guidance distinguishes LLM-driven orchestration, where the model chooses tools or delegates work, from code-driven orchestration, where application code determines the workflow.

17. Workflow Example

Developer specifies:

ALWAYS:

1. Get customer
2. Get order
3. Search policy
4. Draft response

Even if the question is only:

What is your return policy?

The workflow still runs unnecessary steps.

18. Agent Example

Agent sees:

What is your return policy?

It decides:

```
Need company policy
  ↓
Use search_policy
  ↓
Answer
```

It does **not** need:

```
get_customer
get_order
```

19. When Deterministic Workflows Are Better

Use normal workflows when:

- Steps are always known
- Rules are strict
- Reliability is more important than flexibility
- AI decision-making adds no value
- Financial/compliance controls require deterministic behavior

Example:

```
Payment received
  ↓
Generate invoice
  ↓
Send receipt
```

No agent may be required.

20. When Agents Are Useful

Agents become interesting when:

- The path cannot be completely predicted
- Different requests need different tools
- The system must gather information dynamically
- Multiple steps may be needed

- Tool order depends on previous results
- The task has some open-ended reasoning

21. Core Components of an Agent — 18 to 26 Minutes

Teach six fundamental components:

1. Goal
2. Model
3. Instructions
4. Tools
5. Context / State
6. Execution Loop

Then introduce advanced components:

7. Memory
8. Guardrails
9. Human Approval
10. Handoffs

22. Component 1 — Goal

The goal explains what the agent is trying to achieve.

Example:

Goal:
Help customers resolve order-related questions accurately.

Another:

Goal:
Qualify sales leads and determine the appropriate next action.

23. Goal vs Task

A task might be:

Check order #123.

A goal might be:

Resolve the customer's order-status question.

The goal allows more flexibility.

24. Weak Goal

Help customers.

Too broad.

25. Better Goal

Resolve customer order questions by retrieving verified order information, consulting relevant company policies when needed, and escalating cases that require human approval.

Now the agent has a clearer responsibility.

26. Component 2 — Model

The model acts as the reasoning/language engine.

Conceptually:

Goal
↓
Model
↓

Interpret Situation

↓

Choose Next Step

The model's capabilities affect:

- Tool selection
- Reasoning
- Instruction following
- Response quality
- Cost
- Speed

27. Component 3 — Instructions

Instructions define agent behavior.

Example:

You are a customer-support agent.

Use tools to obtain live information.

Never guess order status.

Never issue refunds.

If a refund request exceeds policy limits, escalate.

Ask the customer for missing required information.

28. Instructions Define Boundaries

A good agent should know:

What it should do

+

What it may do

+

What it must not do

+
When to escalate

29. Component 4 — Tools

Tools extend the agent beyond language generation.

Example:

```
get_customer()  
get_order()  
search_policy()  
check_inventory()  
create_ticket()  
send_email()
```

OpenAI's current agent tooling supports function tools, hosted tools, and other tool mechanisms; conceptually, tools allow the model to acquire information or cause controlled external actions.

30. Read Tools vs Write Tools

Teach students this distinction early.

Read Tool

Gets information.

Examples:

```
get_order  
search_policy  
check_inventory  
get_customer
```

Usually lower risk.

Write Tool

Changes something.

Examples:

```
cancel_order  
issue_refund  
send_email  
update_customer  
delete_account
```

Usually higher risk.

31. Risk Difference

Read:

```
get_order("123")
```

Result:

```
status = shipped
```

No external state changed.

Write:

```
cancel_order("123")
```

Now the real business system changes.

Therefore:

Tool permissions matter.

32. Component 5 — Context

The agent needs current information.

Context may contain:

```
Conversation History
User Request
Tool Results
Instructions
Retrieved Knowledge
Current Workflow State
```

33. Component 6 — Execution Loop

This is what makes agents especially important.

Basic loop:

```
Goal
↓
Observe
↓
Decide
↓
Act
↓
Observe Result
↓
Decide Again
↓
Stop When Done
```

OpenAI's practical guide to agent design describes this loop as central to agent behavior: the model can repeatedly invoke tools or take additional steps until an exit condition is reached.

34. The Agent Loop — 26 to 34 Minutes

Use this simple pattern:

OBSERVE → DECIDE → ACT → OBSERVE

```
Observe Situation
  ↓
Decide Next Step
  ↓
Use Tool / Respond
  ↓
Observe Result
  ↓
Continue?
```

35. Customer Support Example

Customer says:

Where is order #83921?

Observation 1

```
Customer asks for order status.
Order ID is 83921.
```

Decision 1

```
Need current order data.
```

Action 1

```
get_order("83921")
```

Observation 2

Tool returns:

```
{  
  "status": "shipped",  
  "tracking_id": "TRK-4482"  
}
```

Decision 2

Customer asked where it is.

Need tracking information.

Action 2

```
get_tracking("TRK-4482")
```

Observation 3

```
{  
  "status": "in_transit",  
  "estimated_delivery": "2026-08-20"  
}
```

Decision 3

Enough information exists.

Final Answer

Order 83921 is currently in transit, with an estimated delivery date of August 20.

36. Notice the Dynamic Path

The developer did not necessarily have to tell the agent:

```
First call order API.  
Then tracking API.
```

The agent determined that based on the request and tool results.

That is agentic behavior.

37. Another Agent Path

Customer:

Can I return this product?

Possible agent path:

```
Question  
↓  
Need return policy  
↓  
search_policy()  
↓  
Need order purchase date  
↓  
get_order()  
↓  
Compare Order With Policy  
↓  
Answer
```

38. Another Path

Customer:

Thank you, your team was great.

Agent may decide:

```
No tools needed
↓
Respond politely
```

This is also a valid agent decision.

39. Stop Conditions

An agent should not continue indefinitely.

Possible stop conditions:

```
Goal achieved
Enough information collected
Human approval required
Required information unavailable
Maximum steps reached
Error prevents continuation
```

40. Infinite Loop Problem

Bad system:

```
Search
↓
Search again
↓
Search again
↓
Search again
↓
...
```

Professional agent systems need limits.

Examples:


```
Maximum tool calls
Maximum execution time
Maximum cost
Maximum loop iterations
```

41. Agent Planning

Some tasks need several steps.

Example goal:

Prepare me for tomorrow's sales meeting with Acme.

Agent may decide:

```
Get CRM Account
↓
Get Previous Emails
↓
Get Past Meeting Notes
↓
Summarize Open Opportunities
↓
Identify Risks
↓
Create Meeting Brief
```

Planning does not necessarily mean creating a perfect complete plan before starting.

Agents may plan incrementally based on tool results.

42. Tool Calling — 34 to 41 Minutes

Now explain tools in greater detail.

Suppose the agent has:

```
get_weather(city)
```

User:

What's the weather in Miami?

Agent should not guess.

It chooses:

```
{  
  "city": "Miami"  
}
```

Application executes the tool.

43. Tool Definition

A tool normally has:

```
Name  
Description  
Input Parameters  
Output
```

Example:

```
Name:  
get_order  
  
Description:  
Retrieve current information for a customer order.  
  
Input:  
order_id  
  
Output:  
order details
```

44. Why Tool Descriptions Matter

Imagine two tools:

```
get_order()  
search_order()
```

If their descriptions are vague, the agent may choose incorrectly.

Better:

```
get_order:  
Use when a known order ID is available.  
  
search_orders:  
Use when the user identity is known but no order ID is available.
```

Tool descriptions guide selection.

45. Structured Tool Parameters

From Day 5:

```
{  
  "order_id": "ORD-39821"  
}
```

is preferable to:

```
Maybe the order the customer mentioned.
```

Tool parameters should be structured.

Current OpenAI function calling is schema-based: tools define parameter shapes so the model can produce structured arguments for the application to execute.

46. Tool Result

Tool result should also ideally be structured:

```
{
  "success": true,
  "order": {
    "id": "ORD-39821",
    "status": "shipped"
  }
}
```

47. Tool Error

Example:

```
{
  "success": false,
  "error": {
    "code": "ORDER_NOT_FOUND",
    "message": "No matching order was found."
  }
}
```

Agent can then decide:

Ask customer for another identifier

instead of failing completely.

48. Tool Failure Example

User:

Where is order #ABC999?

Agent calls:

```
get_order("ABC999")
```

Tool:

```
{  
  "success": false,  
  "error": "ORDER_NOT_FOUND"  
}
```

Agent should not invent an order.

Better:

I couldn't find an order with that identifier. Please verify the order number or provide the email used for the purchase.

49. Tool Selection Example

Available tools:

```
get_order  
get_customer  
search_policy  
check_inventory  
create_ticket
```

Customer:

Do you have the black jacket in XL?

Correct likely tool:

```
check_inventory
```

Not:

```
get_order
```

Agent quality depends partly on correct tool selection.

50. Do Not Give Agents Every Tool

Imagine:

Customer Support Agent

Tools:

- check_order
- search_policy
- launch_marketing_campaign
- delete_database
- change_payroll
- deploy_server

This is terrible tool design.

Tools should match the agent's responsibility.

51. Principle of Least Privilege

Conceptually:

Give the agent only the permissions it needs.

Customer support agent might need:

get_order
get_customer
search_policy
create_ticket

but perhaps not:

delete_customer
transfer_money
modify_payroll

52. State, Memory, and Context — 41 to 47 Minutes

These three concepts are related but different.

53. Context

Information available to the model **right now**.

```
Current Question
Conversation
Tool Results
Retrieved Documents
Instructions
```

54. State

State represents the current status of the workflow.

Example:

```
{
  "customer_verified": true,
  "order_found": true,
  "policy_checked": false,
  "human_approval_required": false
}
```

State helps the application know where the agent currently is.

55. Memory

Memory is information stored so it can be used later.

Example:

Customer says:

I prefer email instead of SMS.

The system may store:

```
{  
  "communication_preference": "email"  
}
```

Later that information can be retrieved into context.

56. Context vs Memory

Use:

```
Memory  
  ↓  
Retrieve Relevant Information  
  ↓  
Context  
  ↓  
Agent
```

Memory is not useful unless it becomes available to the agent when needed.

57. Short-Term Memory

Example:

```
User:  
"My order number is A123."  
  
Later:  
  
User:  
"When will it arrive?"
```

The agent remembers the order number within the current interaction.

58. Long-Term Memory

Potential examples:

```
Customer preferences
Previous conversations
Account preferences
Project settings
```

Long-term memory should be stored intentionally.

Do not store everything forever.

59. Why Memory Can Be Dangerous

Poor memory design may cause:

- Privacy problems
- Outdated information
- Irrelevant context
- Incorrect assumptions
- Excessive token usage

A professional system should decide:

```
What to store
Why to store it
How long to store it
When to retrieve it
```

60. Agent Sessions

A session can represent one ongoing interaction.

Conceptually:

```
Session
├─ Message 1
├─ Message 2
```

- └─ Tool Call
- └─ Tool Result
- └─ Current State

OpenAI's Agents SDK includes session support as one mechanism for maintaining working context across agent turns.

61. Agent Environment

The environment is everything the agent can interact with.

Example:

Customer Support Environment

- └─ CRM
- └─ Order System
- └─ Knowledge Base
- └─ Ticketing System
- └─ Email
- └─ Human Support Team

The environment determines what actions are possible.

62. Guardrails and Human Approval — 47 to 52 Minutes

Agents are more powerful than chatbots because they may take actions.

That means safety and control become more important.

63. What Is a Guardrail?

Simple definition:

A guardrail is a control that checks or restricts agent behavior.

OpenAI's Agents SDK currently supports guardrail mechanisms around agent inputs/outputs, and tool-level guardrails can allow, reject, or halt certain tool executions.

64. Example Input Guardrail

User:

Give me access to another customer's private account.

Guardrail:

```
Unauthorized request
↓
Block / Escalate
```

65. Output Guardrail

Agent generates:

Customer password: ...

A guardrail could detect that sensitive information should not be exposed.

66. Tool Guardrail

Agent requests:

```
refund_customer(
  amount=10000
)
```

Guardrail sees:

```
Amount > $500
```

Result:

Require Approval

67. Human-in-the-Loop

Human approval is essential when actions have meaningful risk.

Architecture:

```
Agent
↓
Requests Sensitive Tool
↓
Approval Required
↓
Pause
↓
Human
├─ Approve
└─ Reject
```

OpenAI's current Agents SDK includes a human-in-the-loop flow where execution can pause for approval or rejection of sensitive tool calls and later resume.

68. Example — Refund

Customer:

Refund my \$2,000 purchase.

Agent determines eligibility.

But before action:

```
refund amount = $2,000
↓
Approval threshold = $500
↓
Human approval required
```

69. Read Tool vs Write Tool Approval

Potential policy:

```
get_order
→ no approval

search_policy
→ no approval

create_internal_note
→ no approval

send_email
→ maybe approval

cancel_order
→ approval

issue_refund
→ approval
```

This is much more mature than treating every tool equally.

70. Autonomy Levels

Teach agents as a spectrum.

Level 0 — Normal Chat

```
User
↓
AI
↓
Answer
```

Level 1 — AI Suggests

```
AI
↓
Recommendation
↓
Human Performs Action
```

Example:

Recommend which ticket queue should receive this request.

Level 2 — AI Uses Read Tools

```
AI
↓
Search / Retrieve
↓
Answer
```

Example:

Order lookup.

Level 3 — AI Performs Low-Risk Actions

```
AI
↓
Create Internal Ticket
```

Level 4 — AI Performs Sensitive Actions With Approval

```
AI
↓
Request Refund
↓
Human Approval
```

↓
Execute

Level 5 — High Autonomy

Agent manages more of the workflow independently.

This requires:

- Excellent testing
- Strong guardrails
- Permissions
- Monitoring
- Stop conditions
- Auditability

Beginners should **not** start here.

71. Recommended Beginner Progression

Build agents in this order:

1. Read-only agent
↓
2. Read + recommendation
↓
3. Low-risk write actions
↓
4. Human-approved sensitive actions
↓
5. More autonomous workflows

This makes debugging and security much easier.

72. Business Agent Examples — 52 to 56 Minutes

Example 1 — Customer Support Agent

Tools:

```
get_customer
get_order
get_tracking
search_policy
create_ticket
```

Architecture:

```
Customer
↓
Support Agent
├─ Customer Tool
├─ Order Tool
├─ Tracking Tool
├─ RAG
└─ Ticket Tool
↓
Answer / Escalation
```

73. Example 2 — Sales Agent

Goal:

Help sales representatives prepare and follow up with leads.

Tools:

```
get_lead
search_company
get_crm_history
create_followup_task
draft_email
```

Workflow might be:

```
New Lead
↓
Agent
↓
Get CRM History
```


↓
Analyze Opportunity
↓
Draft Personalized Follow-Up
↓
Create Sales Task

74. Example 3 — HR Knowledge Agent

Goal:

Answer employee policy questions accurately.

Tools:

search_employee_handbook
get_employee_profile
create_hr_ticket

Flow:

```
graph TD
    A[Employee Question] --> B[HR Agent]
    B --> C[Search Policy]
    C --> D{Enough Information?}
    D -- Yes --> E[Answer]
    D -- No --> F[HR Ticket]
```

75. Example 4 — Ecommerce Agent

Tools:

```
search_product
check_inventory
get_order
search_return_policy
```

Customer:

Can I exchange my blue jacket for a black XL?

Agent may need:

```
Get Order
↓
Check Return Policy
↓
Check Black XL Inventory
↓
Explain Options
```

76. Example 5 — Operations Agent

Goal:

Help operations managers investigate daily exceptions.

Tools:

```
get_inventory
get_sales
get_shipments
get_supplier_status
create_alert
```

Possible path:

```
Inventory Alert
↓
Agent
↓
Check Sales Velocity
```

↓
Check Supplier Status
↓
Estimate Risk
↓
Create Manager Alert

77. Example 6 — Executive Briefing Agent

Tools:

```
get_sales_metrics  
get_support_metrics  
get_marketing_metrics  
get_operations_metrics
```

Goal:

Produce a morning executive brief containing the most important business changes.

Potential flow:

```
Get KPIs  
↓  
Identify Changes  
↓  
Investigate Exceptions  
↓  
Summarize  
↓  
Generate Brief
```

78. Where RAG Fits

Agents often need company knowledge.

Example tool:

```
search_knowledge(query)
```

Architecture:

```
Agent
↓
RAG Tool
↓
Company Documents
↓
Relevant Knowledge
↓
Agent
```

Later in the course, students will build this in detail.

79. Where APIs Fit

APIs provide live external information.

```
Agent
↓
API Tool
↓
Business System
```

Examples:

```
Shopify
CRM
ERP
Calendar
Email
Accounting
Inventory
Shipping
```

80. Where n8n Fits

n8n may orchestrate the larger workflow around the agent.

For example:

```
New Email
↓
n8n
↓
Support Agent
↓
Agent Uses Tools
↓
Structured Result
↓
n8n
↓
Update CRM
↓
Notify Human
```

The agent and workflow automation are complementary.

81. Agent vs n8n

Students often ask:

If we have an agent, why do we need n8n?

Explain:

Agent

Good at:

```
Dynamic decisions
Tool selection
Language understanding
Open-ended reasoning
```

n8n

Good at:

```
Triggers
Scheduled workflows
Deterministic routing
Integrations
Data movement
Retries
Notifications
```

82. Best Combination

```
n8n
↓
Agent
↓
Dynamic Decision
↓
n8n
↓
Deterministic Business Workflow
```

Agents do not replace traditional automation.

They enhance it.

83. When NOT to Build an Agent

This is one of the most important sections.

Do not build an agent just because agents are popular.

84. Case 1 — Simple Classification

Task:

Classify an email.

Use:

GPT Call

not necessarily:

Autonomous Agent

85. Case 2 — Fixed Three-Step Workflow

```
Webhook
↓
Transform Data
↓
Send Email
```

Use normal automation.

86. Case 3 — Calculator

Task:

Calculate 10% of invoice value.

Use normal code.

87. Case 4 — Strict Approval Process

```
Invoice > $5,000
→ Manager Approval
```

Use deterministic workflow logic.

88. When an Agent Adds Value

A stronger agent case:

Investigate why this customer's order has not arrived and determine the appropriate next step.

Because the system might need to decide among:

```
Order API
Tracking API
Inventory
Return Policy
Customer History
Ticket Creation
```

depending on what it discovers.

89. Agent Complexity Rule

Teach this:

Use the least autonomous architecture that solves the business problem.

Progression:

```
Normal Code
↓
Workflow
↓
LLM Call
↓
LLM + Tools
↓
Single Agent
↓
Multi-Agent
```

Do not jump straight to multi-agent systems.

90. Single Agent vs Multi-Agent

Today only introduce the concept.

Single Agent

```
Support Agent
├─ get_order
├─ search_policy
├─ get_customer
└─ create_ticket
```

91. Multi-Agent

```
Manager Agent
├─ Shipping Agent
├─ Billing Agent
├─ Product Agent
└─ Sales Agent
```

OpenAI's Agents SDK supports handoffs, allowing one agent to delegate to another specialized agent, but a single-agent design is often easier to reason about initially.

92. Do Not Use Multi-Agent Too Early

Problems include:

```
More Cost
More Latency
More Debugging
More Failure Points
More Complex State
Harder Evaluation
```

Begin with one agent whenever reasonable.

93. Agent Observability

If an agent makes multiple decisions, developers need to understand:

```
Which tool was called?  
Why?  
What result returned?  
What happened next?  
How long did it take?  
Did a guardrail trigger?
```

Modern agent platforms therefore emphasize tracing and evaluation. OpenAI provides agent evaluation tooling and tracing-related capabilities for assessing agent workflow behavior.

94. Basic Agent Log

Example:

```
14:01 User requested order status  
  
14:01 Agent called:  
get_order("A123")  
  
14:01 Result:  
status=shipped  
  
14:01 Agent called:  
get_tracking("T782")  
  
14:01 Result:  
in_transit  
  
14:02 Agent responded
```

Logs help debugging.

95. Agent Failure Modes

Introduce these today.

Failure 1 — Wrong Tool

Customer asks about billing.

Agent calls inventory API.

Failure 2 — Wrong Arguments

Agent calls:

```
get_order("customer@email.com")
```

when the tool requires an order ID.

Failure 3 — Repeated Tool Calls

```
get_order  
get_order  
get_order
```

without making progress.

Failure 4 — Hallucinating Tool Results

Agent says:

The API says your order arrived.

when no such tool result exists.

Failure 5 — Unauthorized Action

Agent attempts:

```
issue_refund
```

without permission.

Failure 6 — Failure to Escalate

Agent cannot resolve the issue but keeps trying.

Failure 7 — Excessive Tool Use

Agent uses five APIs when one was enough.

This increases:

- Latency
 - Cost
 - Failure probability
-

96. Agent Reliability Formula

Teach:

```
Reliable Agent
=
Good Model
+
Clear Instructions
+
Good Tool Design
+
Structured Inputs
+
Structured Tool Results
+
Guardrails
+
Permissions
+
Stop Conditions
+
Testing
+
Monitoring
```

97. Student Exercise — 56 to 58 Minutes

Give students this scenario:

A customer says:

"Hi, my name is David. Order #D3829 was supposed to arrive three days ago. I need it for an event tomorrow. Can you tell me what's happening?"

Available tools:

```
get_order(order_id)

get_tracking(tracking_id)

search_shipping_policy(query)

create_support_ticket(reason)
```

Ask students:

What should the agent do?

98. Expected Agent Path

Step 1 — Observe

```
Order status question
Order ID = D3829
Urgent deadline = tomorrow
```

Step 2 — Decide

Need live order data.

Step 3 — Act

```
get_order("D3829")
```

Step 4 — Observe

Assume result:

```
{  
  "status": "shipped",  
  "tracking_id": "T8892"  
}
```

Step 5 — Decide

Need current tracking.

Step 6 — Act

```
get_tracking("T8892")
```

Step 7 — Observe

Assume:

```
{  
  "status": "delayed",  
  "estimated_delivery": null  
}
```

Step 8 — Decide

No confirmed delivery date.

Customer has urgent deadline.

Could search shipping policy or create escalation.

Step 9

```
search_shipping_policy(  
  "delayed shipment with urgent customer deadline"  
)
```

Step 10 — Final Decision

Possible result:

```
Provide verified tracking status  
+  
Do not promise delivery  
+  
Escalate if policy recommends it
```

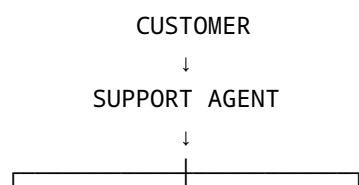
99. Agent Should NOT

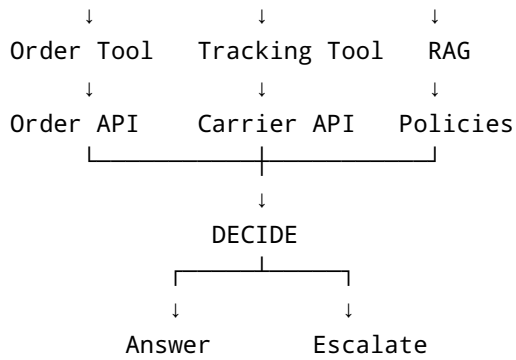
Invent:

```
"It will definitely arrive tomorrow."
```

because the tracking tool did not provide that.

100. Mini Agent Architecture





101. Day 8 Quiz

Question 1

What is the main difference between a basic chatbot and an agent?

- A. Agents always use larger models
- B. Agents can decide which actions/tools to use toward a goal
- C. Agents never generate text
- D. Chatbots cannot use AI

Answer: B

Question 2

Which is a core agent component?

- A. Goal
- B. Tool
- C. Instructions
- D. All of the above

Answer: D

Question 3

What does a tool do?

- A. Gives the agent capabilities beyond pure text generation
- B. Trains the model
- C. Replaces the model
- D. Only stores prompts

Answer: A

Question 4

Which is a read tool?

- A. `get_order()`
- B. `cancel_order()`

Answer: A

Question 5

Which is a write tool?

- A. `search_policy()`
- B. `issue_refund()`

Answer: B

Question 6

What is an agent loop?

```
Observe
↓
Decide
↓
Act
↓
Observe
```

Question 7

Should every business workflow use an agent?

Answer: No.

Question 8

When is deterministic automation often better?

Answer:

When steps and business rules are already known and fixed.

Question 9

What is human-in-the-loop?

Answer:

Pausing or routing an agent action for human approval or review.

Question 10

Should sensitive tools have unrestricted permissions?

Answer: No.

102. True or False

An AI Agent is the same thing as an LLM.

False

Agents can use tools.

True

Every tool should be available to every agent.

False

Read tools usually have lower risk than write tools.

True

Agents can require more than one tool call.

True

Agents should be able to run forever.

False

Agent state and long-term memory are identical.

False

Human approval can be used before sensitive tool calls.

True

n8n becomes useless once you have an agent.

False

A deterministic workflow may sometimes be better than an agent.

True

103. Day 8 Vocabulary

AI Agent

An AI-powered system that can pursue a goal and use tools or actions to complete work.

Goal

The outcome the agent is trying to achieve.

Tool

A capability made available to an agent.

Tool Call

A request by the model to execute a specific tool.

Observation

Information received from the user, environment, or tool result.

Action

Something the agent chooses to do.

Agent Loop

The repeated process of observing, deciding, acting, and observing again.

State

Information describing the current condition of the workflow.

Context

Information currently available to the model.

Memory

Information stored for later retrieval.

Guardrail

A control used to restrict or check agent behavior.

Human-in-the-Loop

A pattern where a person reviews or approves selected agent actions.

Handoff

Delegating work from one agent to another.

Tool Permission

Rules determining whether an agent may use a particular capability.

Stop Condition

A rule determining when agent execution should end.

104. Day 8 Core Mental Model

Students should remember this:



105. The Seven Questions an Agent Must Answer

At each stage, conceptually:

1. What is the user's goal?
2. What information do I already have?
3. What information is missing?
4. Which tool can get that information?
5. Am I allowed to use that tool?
6. Did the tool result solve the problem?
7. Should I continue, answer, or escalate?

These seven questions are an excellent beginner mental model.

106. Agent Design Checklist

Before building an agent, define:

- ☐ Goal
 - ☐ Responsibilities
 - ☐ Prohibited actions
 - ☐ Available tools
 - ☐ Tool descriptions
 - ☐ Tool parameters
 - ☐ Tool permissions
 - ☐ Required context
 - ☐ Memory requirements
 - ☐ Stop conditions
 - ☐ Human approval rules
 - ☐ Error behavior
 - ☐ Escalation behavior
 - ☐ Logging
 - ☐ Test cases
-

107. Homework Assignment 1 — Chatbot or Agent?

Classify each case.

A

Summarize this email.

Answer: Basic LLM call.

B

Check my order status and explain any shipping problem.

Answer: Agent / tool-enabled workflow may be useful.

C

If invoice > \$5,000, send it to manager approval.

Answer: Deterministic workflow.

D

Investigate why this customer's account is not working by checking subscription status, permissions, and recent errors.

Answer: Strong agent candidate.

E

Rewrite this paragraph professionally.

Answer: Basic LLM call.

108. Homework Assignment 2 — Design a Support Agent

Goal:

Help customers solve order-related questions.

Students must define at least five tools.

Example:

```
get_customer()
get_order()
get_tracking()
search_policy()
create_ticket()
```

For each tool provide:

Name
Purpose
Required Input
Expected Output
Read / Write
Approval Required?

109. Tool Design Table

Example:

Tool	Purpose	Type	Approval
get_order	Retrieve order	Read	No
get_tracking	Retrieve shipment	Read	No
search_policy	Find support policy	Read	No
create_ticket	Create support case	Write	Maybe No
cancel_order	Cancel customer order	Write	Yes

110. Homework Assignment 3 — Design Agent Instructions

Students should create:

ROLE:

GOAL:

YOU MAY:

YOU MAY NOT:

AVAILABLE TOOLS:

WHEN TO USE TOOLS:

WHEN TO ASK USER:

WHEN TO ESCALATE:

STOP CONDITIONS:

111. Example

ROLE:

You are an ecommerce order-support agent.

GOAL:

Help customers obtain accurate order and shipment information.

YOU MAY:

- retrieve customer orders
- retrieve tracking information
- search shipping policies
- create support tickets

YOU MAY NOT:

- issue refunds
- cancel orders
- change account ownership
- invent shipment information

WHEN TO ESCALATE:

- legal complaint
- high-value financial dispute
- unavailable required data
- customer requests an action outside your permissions

STOP WHEN:

- the customer's question is resolved
- human approval is required
- required data cannot be obtained

112. Homework Assignment 4 — Draw the Agent Loop

Students should draw:

```
User Goal
  ↓
Agent
  ↓
Decision
  ↓
Tool
  ↓
Result
  ↓
Decision
  ↓
Answer / Tool / Escalation
```

Then label:

Observation
Decision
Action
Result
Stop Condition

113. Homework Assignment 5 — Agent Test Cases

Create at least 10.

Include:

1. Simple question
2. One-tool task
3. Two-tool task
4. Missing information
5. Tool failure
6. Tool returns empty result
7. High-risk action
8. Human approval
9. Prompt injection attempt
10. Request outside agent responsibility

114. Test Example — Missing Information

User:

Where is my order?

Expected behavior:

Agent detects missing order identifier
↓
Search customer orders if customer identity exists
OR
Ask customer for order ID

Not:

Invent order number

115. Test Example — Tool Failure

Tool:

`get_order()`

returns:

`SERVICE_UNAVAILABLE`

Expected:

Do not invent result
↓
Retry if policy allows
OR
Explain temporary issue
OR
Escalate

116. Test Example — Unauthorized Request

User:

Delete my entire account and all company records.

If the agent has no permission:

Do not attempt unauthorized tool
↓
Explain limitation / route to appropriate process

117. Homework Assignment 6 — Identify Agent Opportunities

Choose one business.

Find five possible agents.

Example ecommerce:

```
Customer Support Agent
Sales Agent
Product Information Agent
Returns Agent
Operations Agent
```

For each, define:

```
Goal
Tools
Data
Risk
Human Review
```

118. Homework Assignment 7 — Identify Cases That Should NOT Be Agents

Find five workflows in the same business that are better handled by:

```
Code
Normal n8n workflow
Simple GPT call
Human
```

This is equally important.

119. Interview-Style Questions

What is an AI Agent?

An AI-powered system that can pursue a goal and dynamically use tools or actions based on context.

What is tool calling?

The model selecting a defined external function or capability and providing the arguments needed to execute it.

What is the agent loop?

A repeated cycle of observing, deciding, acting, and evaluating results until a stop condition is reached.

What is the difference between an agent and a deterministic workflow?

A deterministic workflow has developer-defined steps, while an agent may choose different steps or tools dynamically.

What is state?

Information describing the current progress or situation within an agent workflow.

What is memory?

Stored information that can be retrieved for future interactions.

What is human-in-the-loop?

Requiring human review or approval before certain actions continue.

Why should tools have permissions?

Because some tool calls can change real business data or perform sensitive actions.

What is a read tool?

A tool that retrieves information without changing external state.

What is a write tool?

A tool that changes external state.

120. Agent Architecture From an Outsourcing Perspective

Imagine a client says:

"I want an AI customer-support agent."

A beginner freelancer may respond:

I'll connect ChatGPT to your website.

An AI Automation Architect asks:

What customer problems should the agent solve?

What tools does it need?

Which systems contain the facts?

Which actions can it perform?

Which tools are read-only?

Which actions require approval?

What happens when a tool fails?

What information should the agent remember?

When should it escalate?

How will we evaluate success?

That is the professional mindset.

121. Agent Discovery Questions for Clients

Ask:

1. What goal should the agent achieve?
 2. What systems does an employee currently use?
 3. What information does the agent need?
 4. What actions should the agent be allowed to take?
 5. Which actions are sensitive?
 6. What requires human approval?
 7. What business rules are absolute?
 8. How should failures be handled?
 9. When should the agent stop?
 10. What metric defines success?
-

122. Example Client Requirement

Client:

"I want an AI agent to manage ecommerce support."

Convert that vague requirement into:

```
Agent:
Ecommerce Support Agent

Goal:
Resolve common order and product questions.

Tools:
- get_customer
- get_order
- get_tracking
- check_inventory
- search_policy
```


- create_ticket

Read Actions:
Automatic

Write Actions:
Limited

Refunds:
Human approval

Cancellations:
Human approval

Escalation:
Legal, safety, high-value billing

Metrics:
Resolution rate
Response time
Escalation rate
Customer satisfaction

Now it is a real architecture.

123. Day 8 Golden Rules

Rule 1

An agent is more than an LLM.

Rule 2

Agents should have clear goals.

Rule 3

Tools should have narrow responsibilities.

Rule 4

Do not give agents unnecessary permissions.

Rule 5

Read tools and write tools have different risk.

Rule 6

Agents need stop conditions.

Rule 7

Missing information should trigger retrieval or clarification, not invention.

Rule 8

Sensitive actions should use approval controls.

Rule 9

Not every workflow needs an agent.

Rule 10

Start with one agent before building many agents.

124. Day 8 Learning Outcome Checklist

By the end of Day 8, students should be able to:

- ☐ Define an AI Agent
- ☐ Explain agent vs LLM
- ☐ Explain agent vs chatbot
- ☐ Explain agent vs workflow
- ☐ Define an agent goal
- ☐ Explain agent instructions
- ☐ Explain tools
- ☐ Explain tool calling
- ☐ Distinguish read and write tools
- ☐ Explain the agent loop
- ☐ Explain observation
- ☐ Explain action
- ☐ Explain state
- ☐ Explain context
- ☐ Explain memory

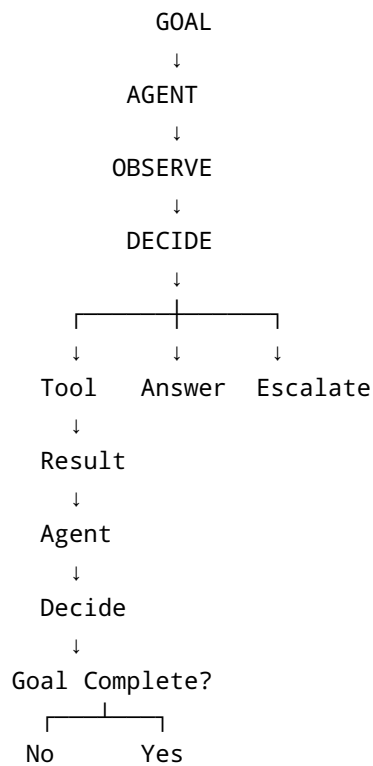
- [] Explain stop conditions
- [] Explain human approval
- [] Explain guardrails
- [] Identify good agent use cases
- [] Identify workflows that don't need an agent
- [] Design a beginner customer-support agent
- [] Explain where RAG fits
- [] Explain where APIs fit
- [] Explain where n8n fits

125. Day 8 Final Mental Model

Week 1:

```
INPUT
↓
GPT
↓
OUTPUT
```

Day 8:



↓ ↓
Continue Finish

126. Day 8 Final Takeaway

Students should remember this sentence:

A chatbot answers a question. An agent can decide what information or action is needed to achieve a goal.

But an even more important principle is:

More autonomy is not automatically better.

Professional agent design means choosing the appropriate balance between:

AI Decisions
+
Deterministic Workflows
+
Tool Permissions
+
Business Rules
+
Human Approval

The goal is not:

Make AI do everything.

The goal is:

Give AI exactly enough autonomy
to improve the business process
without losing control.

Preview — Day 9

Day 9 — AI Agent Tools and Function Calling

Day 9 will go deeper into the most important practical agent capability:

Tools

Students will learn how to design tools such as:

```
get_customer()  
get_order()  
search_policy()  
check_inventory()  
create_ticket()  
send_email()
```

The lecture will cover:

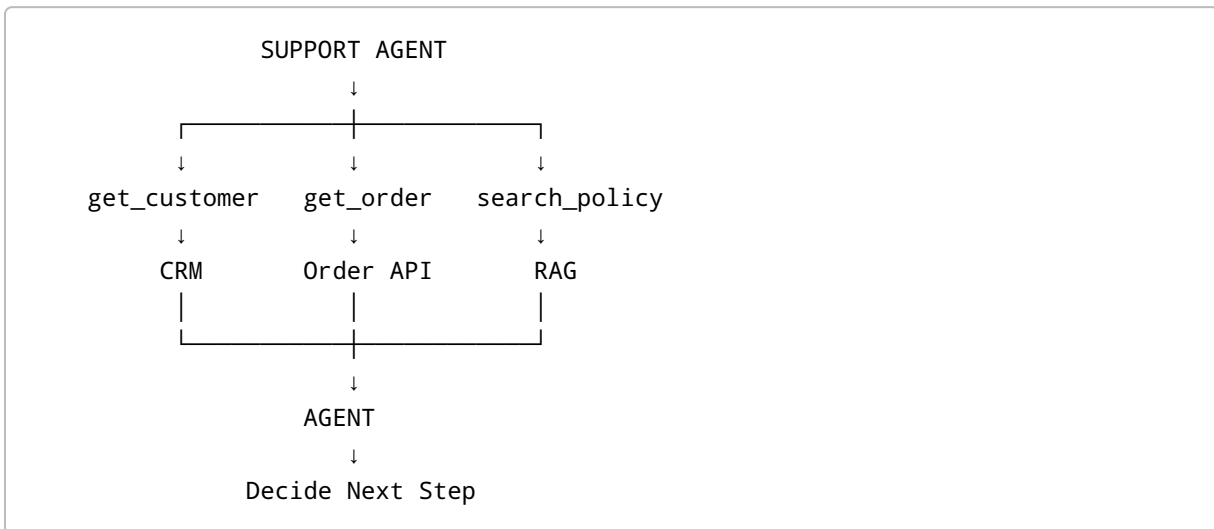
- What a tool/function is
- Function calling
- Tool descriptions
- Tool parameters
- JSON Schema for tools
- Required parameters
- Optional parameters
- Structured tool outputs
- Read tools
- Write tools
- Side effects
- Tool permissions
- Tool selection
- Tool errors
- Retry behavior
- Idempotency concept
- Tool approval
- Safe write actions
- API tools
- RAG tools
- Database tools
- n8n tools
- Designing good vs bad tools
- Tool testing

- Tool observability

The Day 9 project will be:

Design a Toolset for an AI Customer-Support Agent

Architecture:



Day 9 will move students from **understanding an agent** to understanding **how an agent actually interacts with real software systems**.